3 DevOps Weekend Projects

Step-by-Step Build Guide with Azure

Requirements · Steps · Expected Outputs · Practice Tips

Projects Overview

Project 1	Project 2	Project 3
CI/CD End-to-End Deployment	**Docker + Kubernetes Mini Project**	**Terraform + Azure Infrastructure**
Day 1 — CI/CD with Azure DevOps	Day 2 — Kubernetes on Azure AKS	Day 3 — Terraform + Azure VM

PROJECT 1: CI/CD End-to-End Deployment with Azure DevOps

Goal: Automatically build, test, and deploy a Node.js or Python app to Azure App Service — no manual work after a git push.

REQUIREMENTS — What You Need Before Starting

Accounts & Services

- Azure Account (free tier works) — portal.azure.com
- Azure DevOps Account (free) — dev.azure.com
- GitHub Account — github.com
- Docker Hub Account — hub.docker.com

Tools to Install on Your Machine

- Git — git-scm.com
- Docker Desktop — docker.com/products/docker-desktop
- Node.js v18+ OR Python 3.10+ (pick one)
- Azure CLI — learn.microsoft.com/cli/azure/install-azure-cli
- VS Code (recommended IDE)

Knowledge Prerequisites

- Basic Git commands (commit, push, pull, branch)
- Basic understanding of what a Dockerfile is
- Know what a web server / REST API is
- Basic YAML syntax (indentation matters)

STEP-BY-STEP BUILD GUIDE

#	Step / Action	What To Do	Expected Output
1	**Create Your App**	Create a simple Node.js (Express) or Python (Flask) app with 2–3 routes. E.g., GET /health returns {status: ok}. Initialize a Git repo locally.	A running app on localhost:3000 or :5000
2	**Dockerize the App**	Create a Dockerfile at project root. Use official node:18-alpine or python:3.10-slim base image. COPY source, RUN npm install or pip install, EXPOSE port, CMD to start app.	docker build -t myapp . succeeds. docker run -p 3000:3000 myapp works
3	**Push to GitHub**	Create new GitHub repo. git init, git add ., git commit -m 'initial'. Set remote and git push origin main.	Code visible on GitHub at github.com/you/repo

#	Step / Action	What To Do	Expected Output
4	**Create Azure App Service**	In Azure Portal: Create Resource > App Service. Choose Linux, Node.js or Python stack. Free (F1) tier is fine. Note the App Service URL.	App Service URL like https://myapp.azurewebsites.net (blank default page)
5	**Create Azure DevOps Pipeline**	Go to dev.azure.com > New Project > Pipelines > New Pipeline. Connect to GitHub repo. Select 'Starter Pipeline'. This creates azure-pipelines.yml in your repo.	Pipeline created, first run triggered automatically
6	**Write the CI Pipeline (Build + Test)**	Edit azure-pipelines.yml. Add steps: trigger on main, Use Node/Python task, npm install/pip install, Run tests (npm test or pytest), Build Docker image, Push to Docker Hub.	Pipeline green. Docker image pushed to Docker Hub
7	**Add CD Stage (Deploy to Azure)**	Add a Deploy stage in yaml. Use AzureWebApp task or Azure CLI to deploy Docker container to App Service. Add Service Connection in Azure DevOps to authorize.	Deployment stage green in pipeline. App live at Azure URL
8	**Verify Zero-Downtime & Webhooks**	Push a small code change (e.g., change /health response text). Observe pipeline auto-trigger. Verify app updates without downtime using staging slot swap.	New code appears at Azure URL within 2–3 minutes of git push
9	**Add Branch Protection & PR Flow**	In GitHub, set main as protected branch. Require pipeline to pass before merge. Create a feature branch, make a change, open a PR — verify pipeline runs on PR.	PR cannot merge if pipeline fails. Adds real CI enforcement
10	**Add Architecture Diagram + README**	Draw a diagram: GitHub → Azure DevOps Pipeline → Docker Hub → Azure App Service. Write README with setup steps, diagram, pipeline badge.	Clean GitHub repo ready to show in interviews

BONUS: Things the Original Post Missed — Add These

- **Environment Variables & Secrets — Use Azure DevOps Variable Groups for secrets (DB passwords, API keys). Never hardcode in yaml.**
- **Pipeline Caching — Cache node_modules or pip packages between runs to speed up CI from 3 min to 30 sec.**
- **Automated Tests — Add at least unit tests. Pipeline should FAIL if tests fail — that's the whole point of CI.**

- **Notifications — Connect Azure DevOps to email or Slack to get notified on pipeline failures.**
- **Azure Container Registry (ACR) — Instead of Docker Hub, push images to ACR for better security integration with Azure.**

INTERVIEW TOPICS YOU CAN EXPLAIN AFTER THIS PROJECT

- What is a CI/CD pipeline? Walk through your pipeline stages.
- What is a Service Connection and why is it needed?
- How do you handle secrets in pipelines?
- What is zero-downtime deployment and how did you achieve it?
- What triggers your pipeline? How do you prevent bad code from deploying?

PROJECT 2: Docker + Kubernetes Mini Project on Azure AKS

Goal: Containerize a multi-service app (frontend + backend) and orchestrate it using Kubernetes on Azure Kubernetes Service (AKS).

REQUIREMENTS — What You Need Before Starting

Accounts & Services

- Azure Account (free tier — AKS has free cluster management)
- Docker Hub or Azure Container Registry (ACR) account
- GitHub Account

Tools to Install on Your Machine

- Docker Desktop
- kubectl — kubernetes.io/docs/tasks/tools/
- Azure CLI (az command)
- Minikube — for local testing before pushing to AKS
- Helm (optional but recommended) — helm.sh
- VS Code with Kubernetes extension

Knowledge Prerequisites

- Comfortable with Docker (build, run, push)
- Basic YAML syntax
- Know what an API is (frontend calls backend)
- Understand what a port/host is

STEP-BY-STEP BUILD GUIDE

#	Step / Action	What To Do	Expected Output
1	**Build the Backend (Python API)**	Create a simple Python Flask API with endpoints: GET /api/items returns a JSON list. Add a Dockerfile. Test locally: docker build and docker run.	Backend running at localhost:5000. docker ps shows container
2	**Build the Frontend (Nginx)**	Create a simple HTML page that calls the backend API and displays results. Serve it with Nginx. Create Dockerfile using nginx:alpine base image. COPY html files.	Frontend running at localhost:80. Shows data from backend
3	**Test Multi-Container with Docker Compose**	Create docker-compose.yml with both services. Define a shared network. Use service names as hostnames (e.g., backend:5000). Run docker-compose up.	Both containers start. Frontend fetches data from backend via Docker network

#	Step / Action	What To Do	Expected Output
4	**Push Images to Registry**	docker tag and docker push both images to Docker Hub or ACR. Note the image names with tags (e.g., yourdockerid/backend:v1).	Both images visible in Docker Hub or ACR portal
5	**Set Up Local Minikube Cluster**	Install Minikube. minikube start. kubectl get nodes — should show 1 node Ready. Enable ingress addon: minikube addons enable ingress.	minikube status shows Running. kubectl cluster-info works
6	**Write Backend Deployment YAML**	Create backend-deployment.yaml: kind: Deployment, replicas: 2, container image from Step 4, containerPort: 5000. Create backend-service.yaml: kind: Service, type: ClusterIP, port 5000.	kubectl apply -f backend-deployment.yaml succeeds. kubectl get pods shows 2 backend pods Running
7	**Write Frontend Deployment YAML**	Create frontend-deployment.yaml: replicas: 2, nginx image. Create frontend-service.yaml: type: LoadBalancer (or NodePort for local), port 80.	kubectl get pods shows frontend pods Running. kubectl get services shows frontend service with external IP or NodePort
8	**Test Scaling**	kubectl scale deployment backend --replicas=4. Watch pods: kubectl get pods -w. Scale back down to 2. Observe pods terminate gracefully.	4 backend pods appear within ~30 seconds. Downscale removes 2 pods
9	**Perform Rolling Update**	Update backend app code (change a response). Build new Docker image with tag :v2. Push to registry. Update image in deployment: kubectl set image deployment/backend backend=yourid/backend:v2.	Pods update one-by-one. kubectl rollout status shows progress. No downtime during update
10	**Deploy to Azure AKS**	az aks create --name myAKSCluster --resource-group myRG --node-count 2 --generate-ssh-keys. az aks get-credentials to connect kubectl. Apply all YAMLs to AKS cluster.	kubectl get nodes shows Azure VMs. Services have real Azure external IPs. App accessible from internet
11	**Write README + Architecture Diagram**	Draw: Client → LoadBalancer Service → Frontend Pods → ClusterIP Service → Backend Pods. Include all kubectl commands used. Add pod count, image versions.	Professional GitHub repo demonstrating full K8s understanding

BONUS: Things the Original Post Missed — Add These

- **ConfigMaps & Secrets — Store backend URL and API keys as K8s ConfigMap/Secret. Mount them as env variables in pods.**
- **Liveness & Readiness Probes — Add health check probes to deployments. Kubernetes will restart unhealthy pods automatically.**
- **Resource Requests & Limits — Set CPU and memory limits on each container. This is essential for production and great to discuss in interviews.**
- **Horizontal Pod Autoscaler (HPA) — Configure HPA to scale pods based on CPU usage. Demonstrate auto-scaling under load.**
- **Namespaces — Separate dev and prod configs using Kubernetes namespaces.**
- **Persistent Volume — Add a simple database (e.g., PostgreSQL) with a PersistentVolumeClaim to show stateful workloads.**

INTERVIEW TOPICS YOU CAN EXPLAIN AFTER THIS PROJECT

- Difference between a Pod, Deployment, and Service?
- How does Kubernetes service discovery work?
- What is a rolling update and how does Kubernetes do it?
- How would you roll back a bad deployment?
- What is self-healing in Kubernetes?
- Difference between ClusterIP, NodePort, and LoadBalancer?

PROJECT 3: Terraform + Azure Infrastructure as Code

Goal: Provision a complete Azure infrastructure (VNet, Subnet, NSG, VM, install Nginx) using only Terraform code — zero manual clicking in Azure Portal.

REQUIREMENTS — What You Need Before Starting

Accounts & Services

- Azure Account (free tier — VM will cost ~$0.01/hr, destroy after use)
- GitHub Account

Tools to Install on Your Machine

- Terraform CLI v1.5+ — developer.hashicorp.com/terraform/install
- Azure CLI — for authenticating Terraform
- VS Code with HashiCorp Terraform extension
- SSH client (built-in on Mac/Linux, use PuTTY or WSL on Windows)

Knowledge Prerequisites

- Basic understanding of what a VM, VNet, and Subnet are
- Know what SSH is
- Understand what a firewall/security rule is
- Basic command line (cd, ls, cat)

STEP-BY-STEP BUILD GUIDE

#	Step / Action	What To Do	Expected Output
1	**Authenticate Azure CLI**	Run: az login. A browser opens for login. Run: az account show to confirm your subscription. Set: az account set --subscription <id>.	az account show returns your subscription name and ID
2	**Create Project Structure**	Create folder: terraform-azure/. Inside create: main.tf, variables.tf, outputs.tf, terraform.tfvars. This is standard Terraform project layout.	Folder with 4 empty .tf files ready to fill
3	**Configure Terraform Azure Provider**	In main.tf: Add terraform block with required_providers (azurerm). Add provider 'azurerm' block with features {}. Run: terraform init to download provider.	terraform init completes. .terraform/ folder created. Provider plugin downloaded
4	**Create Resource Group**	In main.tf: Add resource 'azurerm_resource_group'. Set name and location (e.g., East	Resource group visible in Azure Portal.

#	Step / Action	What To Do	Expected Output
		US). Run: terraform plan to preview. Run: terraform apply.	terraform.tfstate file created locally
5	**Create Virtual Network & Subnet**	Add resource 'azurerm_virtual_network' (address_space: 10.0.0.0/16). Add 'azurerm_subnet' (address_prefixes: 10.0.1.0/24). Reference RG with resource group name attribute.	VNet and Subnet visible in Azure Portal after terraform apply
6	**Create Network Security Group**	Add 'azurerm_network_security_group'. Add inbound security rules: allow SSH (port 22) and HTTP (port 80) from any source. Associate NSG with subnet.	NSG visible in portal with rules. Subnet shows NSG association
7	**Create Public IP & Network Interface**	Add 'azurerm_public_ip' (allocation_method: Dynamic). Add 'azurerm_network_interface', reference subnet and public IP. Use ip_configuration block.	NIC visible in portal. Public IP allocated (shown as unassigned until VM starts)
8	**Create the Virtual Machine**	Add 'azurerm_linux_virtual_machine'. Set size (Standard_B1s — cheapest). Reference NIC id. Set admin_username. Add admin_ssh_key block with your public SSH key. Set source_image_reference to Ubuntu 22.04.	VM created and running in Azure Portal. SSH access works: ssh adminuser@<public IP>
9	**Install Nginx via Custom Script**	In VM resource, add custom_data block with base64-encoded bash script: #!/bin/bash, apt update, apt install -y nginx, systemctl enable nginx. Destroy and re-apply.	Nginx installed on VM. http://<publicIP> shows Nginx welcome page in browser
10	**Use Variables & Outputs**	Move hardcoded values (location, vm_size, admin_username) to variables.tf. Create variable blocks with type, description, default. In terraform.tfvars, set values. In outputs.tf: output public_ip_address.	terraform apply shows outputs. Running: terraform output public_ip_address returns the IP
11	**Configure Remote State in Azure Blob**	Create Azure Storage Account and Blob Container manually (one-time). Add backend 'azurerm' block in main.tf: resource_group_name, storage_account_name,	terraform.tfstate no longer stored locally. State stored in Azure Blob Storage — team-safe

#	Step / Action	What To Do	Expected Output
		container_name, key. Run: terraform init -reconfigure.	
12	**Destroy Everything**	Run: terraform destroy. Type yes to confirm. Verify all resources deleted in Azure Portal. No lingering charges.	All Azure resources deleted. Azure Portal shows empty resource group (then delete that too)

MODULARIZE YOUR CODE (Next Level)

Once the basic project works, refactor into Terraform Modules — this is what interviewers love to ask about.

- Create folder structure: modules/networking/, modules/compute/, modules/security/
- Each module has its own main.tf, variables.tf, outputs.tf
- Root main.tf calls modules using module blocks and passes variables
- This mirrors real enterprise Terraform codebases

BONUS: Things the Original Post Missed — Add These

- **Terraform Workspaces — Use workspaces to manage dev/staging/prod environments from the same code. terraform workspace new dev.**
- **Data Sources — Use data 'azurerm_client_config' to dynamically fetch tenant/subscription IDs instead of hardcoding.**
- **Count & For Each — Create multiple VMs or resources dynamically using count = 3 or for_each on a map of environments.**
- **Terraform Cloud (Free Tier) — Use Terraform Cloud as your remote backend and get free remote runs, history, and team collaboration.**
- **Tagging Strategy — Add tags to every resource (environment, owner, project). This is mandatory in real enterprise infra.**
- **Azure Key Vault Integration — Store VM admin password in Azure Key Vault and reference it in Terraform using data source.**

INTERVIEW TOPICS YOU CAN EXPLAIN AFTER THIS PROJECT

- What is the difference between terraform plan and terraform apply?
- What is state file and why should it be stored remotely?
- How do you handle secrets in Terraform without exposing them in code?
- What are Terraform modules and why do you use them?
- What happens if two people run terraform apply at the same time?
- How do you import existing Azure infrastructure into Terraform?

Weekend Execution Plan

Day	Project	Morning (4 hrs)	Afternoon (3 hrs)
Day 1	CI/CD Pipeline	Steps 1–5: App + Docker + GitHub + App Service	Steps 6–10: Pipeline YAML + CD + README
Day 2	Docker + Kubernetes	Steps 1–5: Frontend + Backend + Minikube local	Steps 6–11: Deploy to AKS + Scale + README
Day 3	Terraform + Azure Infra	Steps 1–7: Auth + VNet + NSG + NIC	Steps 8–12: VM + Nginx + Remote State + Destroy

After All 3 Projects — Final Checklist

- All 3 projects pushed to GitHub with clean README and architecture diagrams
- Each README has: What the project does, Tools used, How to run it, Screenshots of working output
- Azure resources destroyed after each project (avoid charges)
- Practice explaining each project out loud in 2 minutes — mock interview style
- Add all 3 GitHub links to your LinkedIn profile / resume

Real DevOps starts with hands-on building. Push everything. Learn by doing.